Reflection on Building My Snake Game with a Custom AI Coding Tutor

Mohsin Saghir · WOH AI assignment · github.com/mo-hsin20/snake-game

1. Setup and Agent Configuration

I started by opening my Snake game project in VS Code and using the Chat panel. I selected my custom agent called `snake-tutor`, which I configured to help me build the game step by step instead of giving me complete solutions.

One problem I had during setup was a hidden space in the folder name. I fixed it by checking and correcting the folder name.

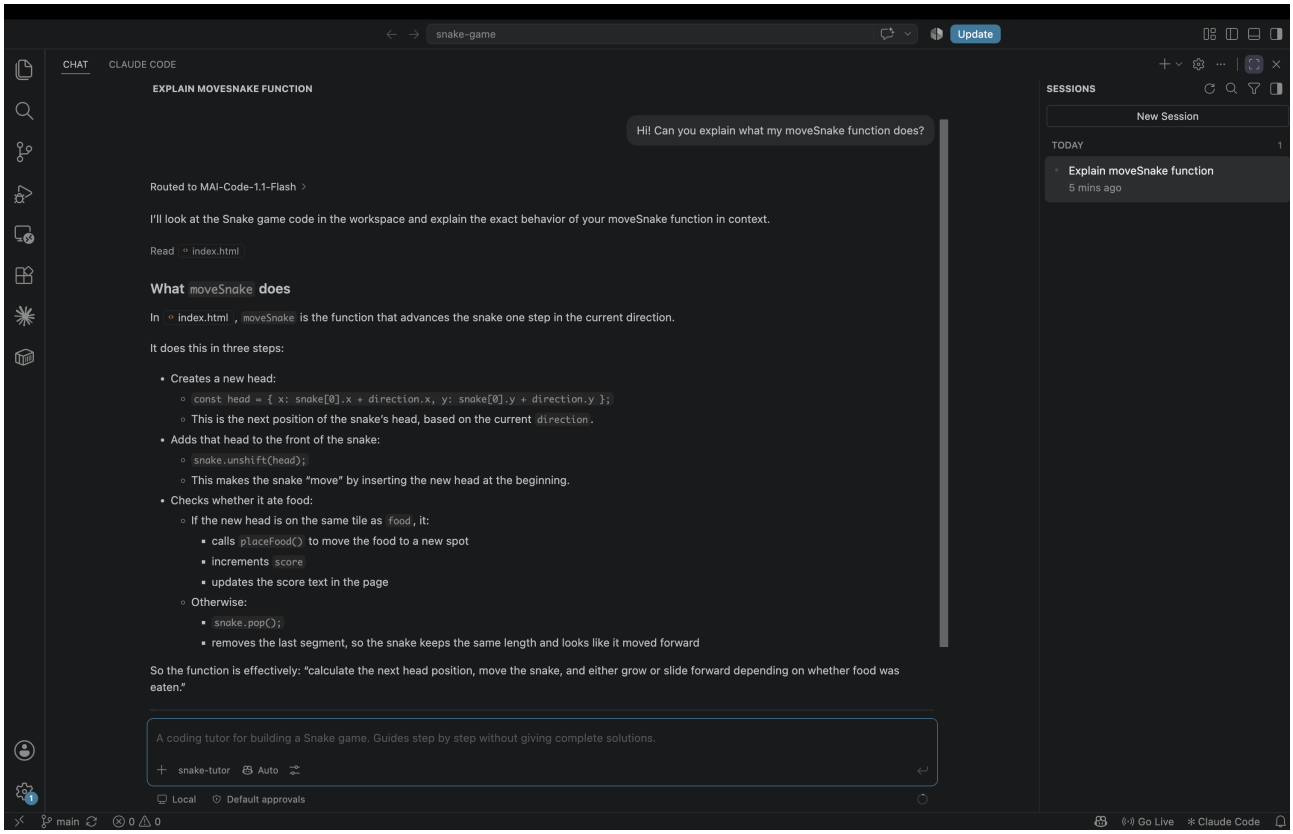
I customized the agent file by adding my current progress, learning goals, and preferences. I explained which features I had already implemented and what I wanted to learn next.

I added the rule "let me choose my own colours" because I wanted to make my own design decisions and learn CSS instead of letting AI decide everything.

I also added the rule "explain errors before fixes" because I wanted to understand why something was not working before receiving a solution. My goal was to learn programming, not just copy code from AI.

2. Development Process

Moment 1: Explaining the moveSnake Function



Screenshot 1: The snake-tutor agent explains my moveSnake function.

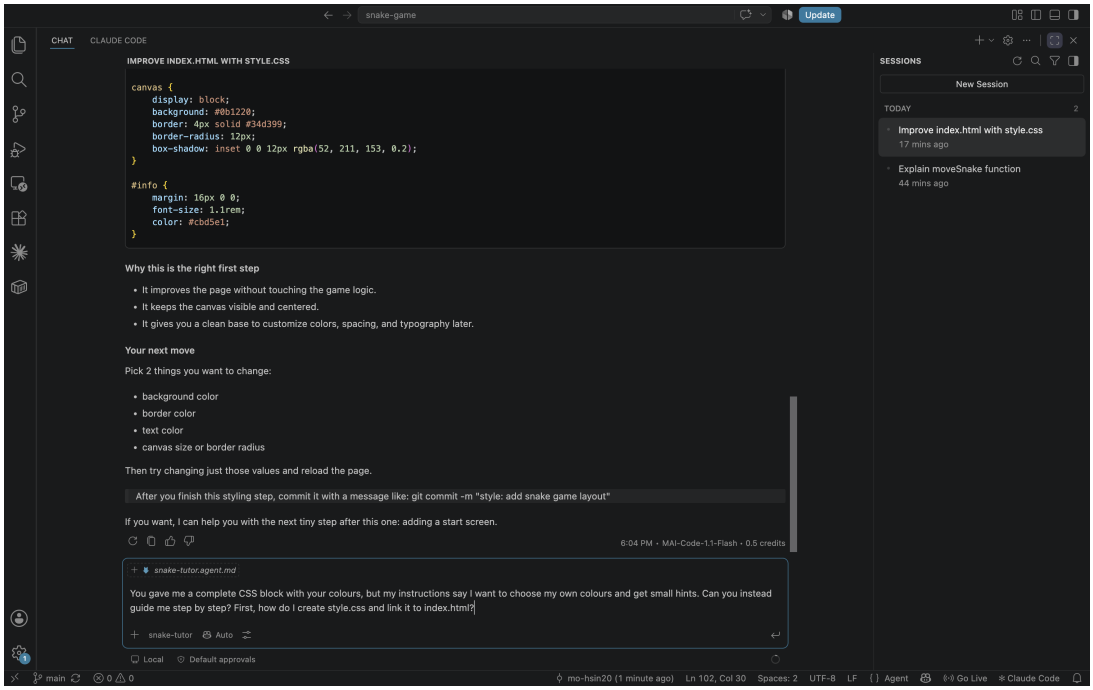
I asked my coding agent to explain what my `moveSnake()` function does. The agent read my `index.html` file and explained the function step by step.

I found this helpful because I could understand my own code better. Instead of just receiving code, I learned how the snake moves and how its length changes when it eats food.

Moment 2: The CSS Block

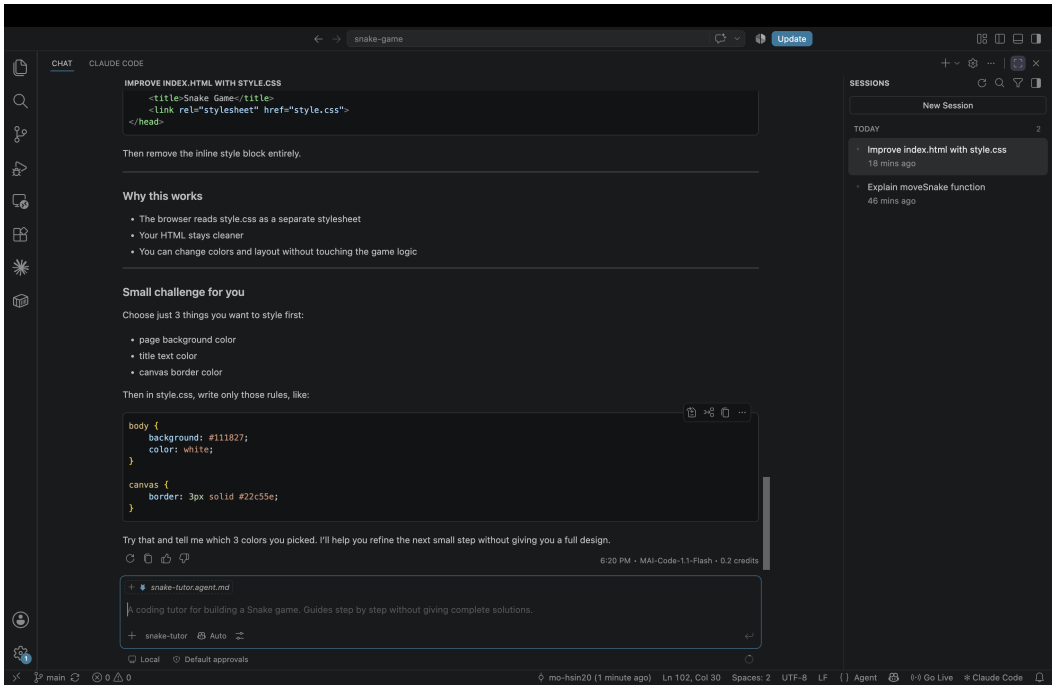
I asked the agent to help me improve the design of my game using CSS. However, the agent initially provided a complete CSS block with its own colours.

This was against my configuration rule because I wanted to choose my own colours and learn step by step.



Screenshot 2: I remind the agent of my rules after it gave me a complete CSS block.

I explained this to the agent and asked it to guide me instead of giving me the complete solution. After that, it showed me how to create a separate `style.css` file and connect it to `index.html`.



Screenshot 3: After my correction, the agent gives a small challenge and lets me pick the colours.

I chose black as the background colour, green for the snake, and red for food because I liked the contrast and wanted a simple gaming design.

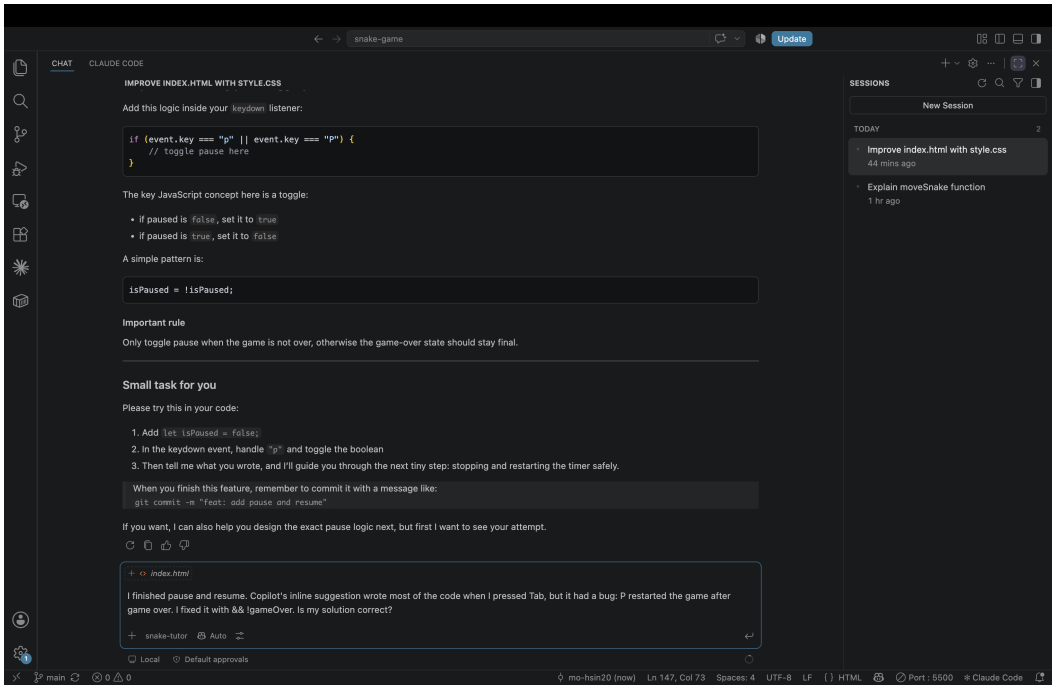
This experience taught me that a custom agent does not always follow my instructions perfectly, and sometimes I need to remind it about my preferences.

Moment 3: Pause and the Inline Suggestion Bug

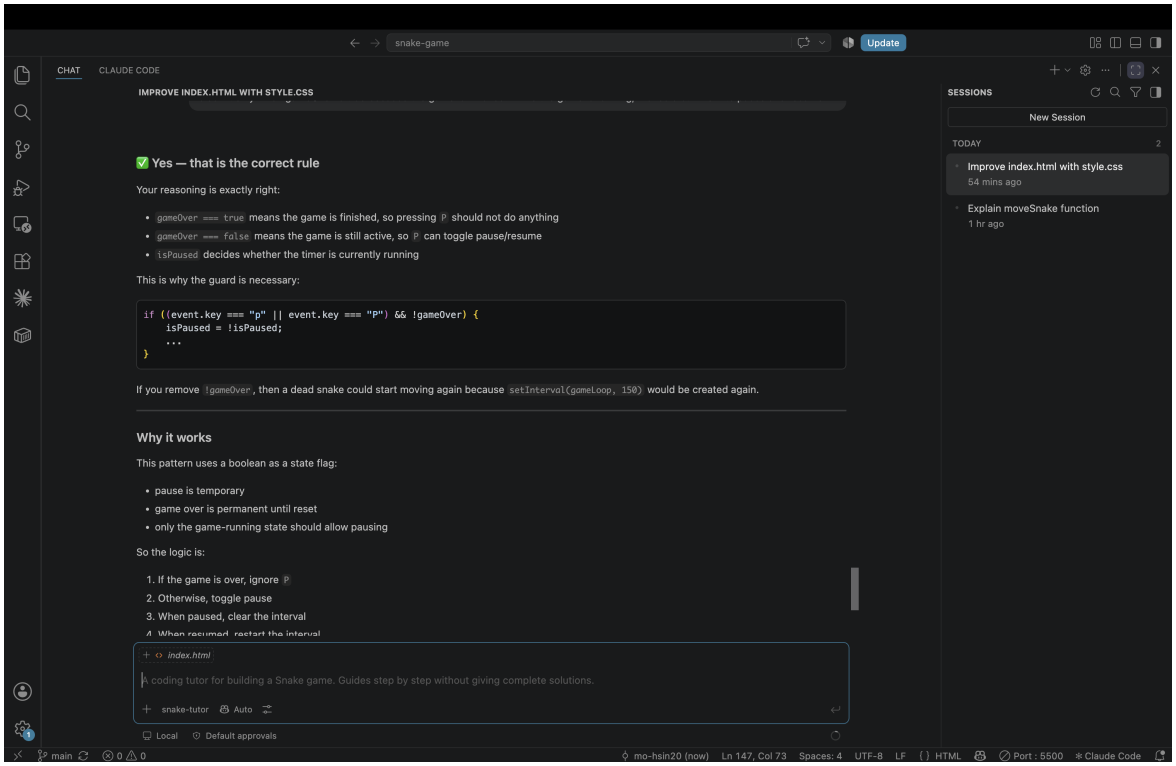
While developing the game, I also worked with inline code suggestions. When I typed the pause-related code, grey ghost text appeared in the editor. I pressed Tab to accept the suggestion.

I noticed a problem with the game logic: the pause functionality needed to work correctly without interfering with the game-over state.

I fixed this by using the condition `&& !gameOver`. This ensured that the game would not resume or continue running when the game was already over.



Screenshot 4: I tell the tutor about the inline suggestion bug and my fix.



Screenshot 5: The tutor confirms my reasoning about the !gameOver guard.

3. Reflection

My agent configuration made the AI more focused on teaching me instead of simply writing code for me. However, sometimes it still provided too much code or suggestions that did not match my preferences.

During this project, I used several JavaScript concepts:

- **Functions:** I used functions such as `moveSnake()` to organize the game logic into smaller tasks.

- **Arrays:** I used arrays to store the snake's body parts and update its position.
- **setInterval:** I used it to make the snake move automatically at regular time intervals.
- **localStorage:** I used it to save the high score in the browser so that it remained available after refreshing the page.

One concept that was new and interesting for me was `localStorage`. I learned that JavaScript can save information in the browser without needing a database. The AI helped me understand how to save and retrieve the high score.

The chat agent and inline suggestions are different. The chat agent can understand my questions, explain code, and guide me through programming tasks. Inline suggestions provide code predictions while I am typing.

The agent was sometimes unhelpful when it gave me complete solutions instead of small hints. I handled this by reminding it about my configuration and asking it to explain things step by step.

Next time, I would improve my agent configuration by adding more rules about giving small hints, asking me questions, and allowing me to solve problems independently before showing possible solutions.

I think Copilot with a custom agent is an **AI coding assistant with some agent-like behaviour** because it can understand my instructions, read project files, and help me with development tasks. My configuration gave me more control over how the AI helped me, especially regarding my learning preferences and coding decisions.

I also used another AI, Claude, as a helper during much of the game development. However, I still needed to check the suggestions and understand the code before accepting changes.

Overall, this project helped me improve my JavaScript, CSS, Git, and problem-solving skills. More importantly, I learned that AI is most useful when I use it as a learning assistant rather than depending on it to write the entire program for me.